Guía Práctica 4 – Complejidad

1. Mostrar usando la definición de $\mathcal{O}()$ tomada de [1] [2] :

$$\mathcal{O}(f(n)) = \{\ t : \mathbb{N} \to \mathbb{R}^+ \mid \exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N} : \forall n \geq n_0 \Rightarrow t(n) \leq cf(n)\ \}.$$

   a) $t(n) = 50n - 8 \in \mathcal{O}(n)$

   b) $t(n) = 5n^2 - 3n + 6 \in \mathcal{O}(n^2)$

   c) $t(n) = 2^n + 7n + 3 \in \mathcal{O}(2^n)$

2. Determinar $\mathcal{O}()$ y $\Omega()$ de las siguientes funciones:

   a) $t(n) = 5n^2$

   b) $t(n) = 3n^2 + 2n$

   c) $t(n) = 3^n + 2^n$

   d) $t(n) = n + \log_2(n)$

   e) $t(n) = n^2 + \log_2(n)$

3. Determinar $\Theta()$ de las siguientes funciones:

   a) $t(n) = n^2 + 2n$

   b)

$$t(n) = \begin{cases} 2n + 4 & \text{n } par \\ 7n^2 + n & \text{n } impar \end{cases}$$

   c)

$$t(n) = \begin{cases} 4n + 5 & \text{n } par \\ 5n + 0.5n & \text{n } impar \end{cases}$$

4. Reescribir los siguientes órdenes de complejidad de menor a mayor ($<$):
   $\mathcal{O}(n), \mathcal{O}(n \log n), \mathcal{O}(n!), \mathcal{O}(n^2), \mathcal{O}(5), \mathcal{O}(\log \log n), \mathcal{O}(3^n), \mathcal{O}(\sqrt{n}), \mathcal{O}(n^n),$
   $\mathcal{O}(n^3), \mathcal{O}(2^n), \mathcal{O}(\log n)$.

   Etiquete cada $\mathcal{O}$ como **sublineal**, **lineal** o **superlineal**. Dentro de éstas últimas puede decir cuáles son **polinómicas** (cuadráticas, cúbicas, etc.) o **exponenciales**.

5. Para cada uno de los fragmentos de código siguientes:
   - Diga qué $\mathcal{O}()$ poseen.
   - Impleméntelos y dé el tiempo de ejecución para $\neq$ valores de n.
   - Compare estimación teórica (primer ítem) con tiempos reales (segundo ítem).

[1] Marzal y Gracia. Introducción al análisis de algoritmos.
[2] https://arco.esi.uclm.es/public/mirror/Introducci%C3%B3n%20al%20an%C3%A1lisis%20de%20algoritmos.pdf

Algorithm 1:

```
sum ← 0
for i ← 1 to n do
|   sum ← sum + 1
end
```

Algorithm 2:

```
sum ← 0
for i ← 1 to n do
|   for j ← 1 to n do
|   |   sum ← sum + 1
|   end
end
```

Algorithm 3:

```
sum ← 0
for i ← 1 to n do
|   for j ← 1 to n^2 do
|   |   sum ← sum + 1
|   end
end
```

Algorithm 4:

```
sum ← 0
for i ← 1 to n do
|   for j ← 1 to i do
|   |   sum ← sum + 1
|   end
end
```

Algorithm 5:

```
sum ← 0
for i ← 1 to n do
|   for j ← 1 to i^2 do
|   |   for k ← 1 to j do
|   |   |   sum ← sum + 1
|   |   end
|   end
end
```

Algorithm 6:

```
i ← 1
while i ≤ n do
|   j ← 1
|   while j ≤ n do
|   |   j ← j * 2
|   end
|   i ← i + 1
end
```

Algorithm 7:

```
i ← 1
while i ≤ n do
|   j ← 1
|   while j ≤ i do
|   |   j ← j + 1
|   end
|   i ← i + 1
end
```

Algorithm 8:

```
i ← 1
while i ≤ 10 do
|   j ← 1
|   while j ≤ 10 do
|   |   j ← j + 1
|   end
|   i ← i + 2
end
```

6. ¿Cuál es el propósito de los siguientes algoritmos? Analizar el tiempo de ejecución en el peor caso y expresarlos en notación $\mathcal{O}$.

Algorithm 9:

Input: a, n: **integer**
Output: b
b: **long integer**
$b \leftarrow 1$
for $k \leftarrow 0$ ***to*** $k < n$ **do**
  $b \leftarrow b * a$
end

Algorithm 10:

Input: a, n: **integers**
Output: b
b: **long integer**
c: **long integer**
$b \leftarrow 1$
$c \leftarrow a$
for $k \leftarrow n$ ***to*** $k > 0$ **do**
  if k ***mod*** $2 == 0$ **then**
    $k \leftarrow k/2$
    $c \leftarrow c * c$
  end
  else
    $k \leftarrow k - 1$
    $b \leftarrow b * c$
  end
end

7. En cada uno de los siguientes fragmentos de código encontrar el error del algoritmo o completar lo que falta, según corresponda, para que realice la tarea intencionada

(a) Given array of integers, find the number of sorted pairs formed by its elements such that one element in the pair is divisible by the other one.

Example

For *sequence = [1, 3, 2]*, the output should be *divisorsPairs(sequence) = 2*.

These pairs are: *(1, 3), (1, 2)*.

```
int divisorsPairs(std::vector<int> sequence) {

  int result = 0;

  for (int i = 0; i < sequence.size(); i++) {
    for (int j = i + 1; j< sequence.size(); j++) {
      if (sequence[i] % sequence[j] == 0 && sequence[j] % sequence
    [i] == 0) {
        result++;
      }
    }
  }

  return result;
}

```

```
16
```

(b) Identify the largest integer of the given length.

Example

For $n = 2$, the output should be *largestNumber(n) = 99*.

```
int largestNumber(int n) {

  int res = 0;

  for (int i = 0; i < n; i++) {
    res = res + 9;
  }

  return res;
}

```

(c) Given array of integers *sequence* and some integer *fixedElement*, output the number of even values in *sequence* before the first occurrence of *fixedElement* or *-1* if *fixedElement* is not contained in sequence.

Example

For *sequence = [1, 4, 2, 6, 3, 1]* and *fixedElement = 6*, the output should be *evenNumbersBeforeFixed(sequence, fixedElement) = 2*.

```
int evenNumbersBeforeFixed(std::vector<int> sequence, int
    fixedElement) {

  int result = 0;

  for (int i = 0; i < sequence.size(); i++) {
    if ( ... ) { //complete the argument of if
      result++;
    }
    if (sequence[i] == fixedElement) {
      return result;
    }
  }

  return -1;
}

```

(d) Given an array of integers, find the rightmost round number (it ends with one or more zeroes) in it and output its position in the array (0-based). If there are *no round numbers* in the given array, output *-1*.

Example

For *inputArray = [0, 5, 10, 15]*, the output should be *rightmostRoundNumber(inputArray)* = 2;

```
int rightmostRoundNumber(std::vector<int> inputArray) {

  int ans = -1;
  for (int i = 1; i < inputArray.size(); i++) {
    if (inputArray[i] % 10 == 0) {
      ans = i;
    }
  }

  return ans;
}
```

(e) Given an array of integers *inputArray* and an integer *maxSum*, for each i ($0 \leq i <$ *length of inputArray*) find the rightmost j ($j <$ *length of inputArray*) such that the sum of all elements between the ith and j^{th} *elements* (both inclusive) is not greater than *maxSum*.

Example

For *inputArray* $= [2, 3, 0, 1, 2]$ and *maxSum* $= 4$, the output should be maximalAllowableSubarrays(inputArray, maxSum) $= [0, 3, 4, 4, 4]$.

```
std::vector<int> maximalAllowableSubarrays(std::vector<int>
    inputArray, int maxSum) {
  std::vector<int> right(inputArray.size());
  int j = -1;
  int curSum = 0;
  for (int i = 0; i < inputArray.size(); i++) {
    if (i > 0) {
      curSum -= inputArray[i - 1];
    }
    while (j + 1 < inputArray.size() && curSum + inputArray[j + 1]
    < maxSum) {
      curSum += inputArray[++j];
    }
    right[i] = j;
  }

  return right;
}
```

8. A partir de una secuencia de números A, calcular otra B tal que cada uno de sus elementos B_i sea el promedio de todos los anteriores en la secuencia original, incluído el $i-$ésimo.
   Ejemplo:
   $A = [1, 3, 5, 4, 2, 1]$
   $B = [1, 2, 3, 3.25, 3, 2.66]$

9. Un arreglo A contiene $n-1$ enteros **únicos** $\in [0, n-1]$; es decir, hay un número de este intervalo que no está en A. Escriba un algoritmo para encontrar ese número

faltante en $\mathcal{O}(n)$. Sólo es posible utilizar $\mathcal{O}(1)$ espacio adicional además del arreglo A mismo.
Ejemplo:
$N = 5$. El arreglo puede contener elementos (no repetidos) $\in [0, 4]$.
Si el vector $= [1, 4, 2, 0]$, entonces el elemento que falta es **3**.

10. Suponga que cada renglón de un arreglo $A_{n,n}$ posee valores 0 y 1 de tal manera que en cualquier renglón de A todos los 1's van antes que los 0's. Describa un método de $\mathcal{O}(n)$ para determinar el renglón de A que contenga la mayor cantidad de 1's.

    Ejemplo:

1	1	0	0	0
1	1	1	0	0
1	**1**	**1**	**1**	**0**
1	0	0	0	0
1	1	1	0	0

11. Para cada solución que tenga de los problemas de las guías anteriores, comente, si es posible, el $\mathcal{O}()$ de su solución. Si no es posible (por ejemplo talla indeterminada del problema, o caminos de ejecución no determinísticos) explique por qué. Para los problemas que no tenga resuelto, trate de proponer un algoritmo (al menos en forma coloquial) y vea si puede determinar el $\mathcal{O}()$ de su propuesta.

12. Realice algoritmos (no es necesario implementarlos, suficiente con pseudocódigo claro) para:

    a) **Sumar** $N \in \mathbb{N}$ y $M \in \mathbb{N}$ tales que n y m representan la cantidad de dígitos cada uno. Piense que el número de dígitos puede ser muy grande.

    b) **Multiplicar** $N \in \mathbb{N}$ y $M \in \mathbb{N}$ tales que n y m representan la cantidad de dígitos cada uno. Piense que el número de dígitos puede ser muy grande.

    c) **Multiplicar** las **matrices** $A_{m,n}$ y $B_{n,q}$.

    NOTA: En todos los casos exprese el $\mathcal{O}()$ de su algoritmo.

13. Resolver en lenguage C++ y obtener **ACCEPTED** en los jueces correspondientes para los siguientes problemas:

    a) Integer Inquiry – http://uva.onlinejudge.org/external/4/424.html

    b) 500! – http://uva.onlinejudge.org/external/6/623.html

    c) Overflow – http://uva.onlinejudge.org/external/4/465.html

    NOTA: En todos los casos exprese el $\mathcal{O}()$ de su algoritmo.